Riassunti e Appunti di Algoritmi

Appunti di corso



Università di Catania

Università degli Studi di Catania
Dipartimento di Matematica e Informatica
Corso di Laurea Magistrale in Informatica (LM-18)

Autore
Emanuele Galiano

Anno accademico
Anno Accademico 2026/2027

Versione: 25 ottobre 2026

Prefazione

Queste dispense sono nate come appunti personali per il corso di *Algoritmi* tenuto presso il Dipartimento di Matematica e Informatica dell'Università di Catania nell'anno accademico 2026/2027. L'obiettivo principale di questo materiale è stato fornire una risorsa personale di studio sulla parte teorica del corso, raggruppando concetti chiave, definizioni, teoremi, immagini ed esempi in un unico documento, e integrando gli appunti presi a lezione con le dispense fornite dal docente.

Questo file non deve essere visto come un testo ufficiale o completo sull'argomento, in quanto potrebbero esserci errori, omissioni o imprecisioni. Invito pertanto chi legge questo documento a consultare le dispense ufficiali del corso proposte dal docente ed eventuali testi di riferimento consigliati. Inoltre, invito chiunque noti errori o abbia suggerimenti a contattarmi via email:

- **Email personale**: galianoo.emanuele@gmail.com,
- **Email universitaria**: emanuele.galiano@studium.unict.it;

oppure se ancora presente online, aprire una **issue** o una **pull request** nel repository GitHub associato a queste dispense:

`https://github.com/emanuelegaliano/Algoritmi-lm18`

In particolare, all'interno del repository GitHub sono presenti il codice open-source dei sorgenti LaTeX utilizzati per generare queste dispense, oltre a tutti i file di supporto come immagini e codici per la generazione di alcuni grafici presenti nel testo.

Licenza

Questo documento, intitolato *Riassunti e Appunti di Algoritmi – Appunti di corso*, è distribuito sotto licenza **Creative Commons Attribution–ShareAlike 4.0 International (CC BY-SA 4.0)**.

La presente licenza è stata scelta con l'obiettivo di favorire la diffusione, la condivisione e il riutilizzo del materiale didattico, garantendo al contempo il riconoscimento degli autori originali e la preservazione della natura open delle opere derivate.

Autori: Emanuele Galiano
Affiliazione: Università degli Studi di Catania – Dipartimento di Matematica e Informatica
Anno accademico: Anno Accademico 2026/2027

Diritti concessi

In conformità con i termini della licenza Creative Commons BY-SA 4.0, è consentito a chiunque di:

- copiare e ridistribuire il materiale in qualsiasi mezzo o formato;
- adattare, modificare e trasformare il materiale;
- utilizzare il materiale anche per scopi commerciali.

Tali diritti sono concessi a titolo gratuito e non possono essere revocati, purché siano rispettate le condizioni indicate nella sezione seguente.

Condizioni

L'utilizzo del materiale è subordinato al rispetto delle seguenti condizioni:

- **Attribuzione (BY):** deve essere fornita un'adeguata attribuzione dell'opera, citando **titolo**, **autori** e, ove possibile, la **fonte**. L'attribuzione deve essere effettuata in modo ragionevole e non tale da suggerire che gli autori originali approvino l'uso o le modifiche apportate.

- **Condividi allo stesso modo (SA):** nel caso in cui il materiale venga modificato, trasformato o utilizzato per creare opere derivate, tali opere devono essere distribuite sotto la *stessa licenza* Creative Commons Attribution–ShareAlike 4.0 International.

Non è consentito applicare termini legali o misure tecnologiche che limitino giuridicamente altri utenti dall'esercitare i diritti concessi dalla licenza.

Assenza di garanzia

Il materiale è fornito *"così com'è"*, senza garanzie di alcun tipo, esplicite o implicite. In particolare, gli autori non garantiscono l'accuratezza, la completezza o l'assenza di errori nel contenuto del documento e declinano ogni responsabilità per eventuali danni derivanti dall'uso del materiale.

Testo completo della licenza

Il testo legale completo della licenza Creative Commons Attribution–ShareAlike 4.0 International è disponibile al seguente indirizzo:

`https://creativecommons.org/licenses/by-sa/4.0/`

Copyright © 2026
Emanuele Galiano

Indice

Licenza **v**

Diritti concessi v
Condizioni v
Assenza di garanzia vi
Testo completo della licenza vi

1 Analisi ammortizzata **1**

1.1 L'idea di fondo 1
1.2 Due esempi di riferimento 2
1.2.1 Stack con MultiPop 2
1.2.2 Contatore binario con Increment 4
1.3 Metodo dell'aggregazione 5
1.3.1 Stack con MultiPop 5
1.3.2 Contatore binario con Increment 7
1.3.3 Contatore non necessariamente nullo 8
1.4 Metodo degli accantonamenti 10
1.4.1 Stack con MultiPop 11
1.4.2 Contatore binario con Increment 12
1.5 Metodo del potenziale 13
1.5.1 Stack con MultiPop 15
1.5.2 Contatore binario con Increment 15
1.5.3 Configurazione iniziale non nulla 16
1.6 Confronto tra i metodi 17
1.7 Tabelle dinamiche 17
1.7.1 Inserimenti con espansione 18
1.7.2 Analisi con l'aggregazione 19
1.7.3 Analisi con gli accantonamenti 20
1.7.4 Analisi con il potenziale 21
1.7.5 Inserimenti e cancellazioni 22

Capitolo 1

Analisi ammortizzata

Quando si studia una struttura dati si è soliti calcolare il costo di ogni singola operazione nel caso peggiore, e da quello ricavare il costo di una sequenza di operazioni. Il problema di questo approccio è che molte strutture dati hanno operazioni che sono occasionalmente molto costose ma che, proprio per questo, lasciano la struttura in uno stato in cui le operazioni successive costano poco: l'analisi del caso peggiore della singola operazione non tiene conto di questo effetto e finisce per fornire stime troppo pessimistiche. L'**analisi ammortizzata** (amortized analysis) studia invece il costo di un'intera sequenza di operazioni e lo ripartisce in qualche modo tra le operazioni che la compongono. A differenza dell'analisi del caso medio non interviene alcuna probabilità, e si ottiene comunque una garanzia sul caso peggiore. In questo capitolo vediamo i tre metodi principali dell'analisi ammortizzata (aggregazione, accantonamenti e potenziale), applicandoli a due esempi: uno stack con l'operazione aggiuntiva MultiPop e un contatore binario con l'operazione Increment. Infine li usiamo per studiare un'applicazione reale, le *tabelle dinamiche*, ovvero tabelle che vengono riallocate quando si riempiono o si svuotano troppo.

1.1 L'idea di fondo

Consideriamo una struttura dati su cui si esegue una sequenza di n operazioni. Ciò che ci interessa non è il costo della singola operazione, ma il tempo complessivo della sequenza, che indichiamo con $T(n)$. Questo tempo viene poi ripartito in qualche modo tra le n operazioni: la scelta più semplice è dividerlo in parti uguali.

> **Definizione 1.1 (Costo ammortizzato per operazione).** Sia $T(n)$ il costo totale, nel caso peggiore, di una sequenza di n operazioni su una struttura dati. Il **costo ammortizzato per operazione** è
>
> $$\frac{T(n)}{n}.$$

Si tratta quindi del costo che ogni operazione ha *in media* all'interno della sequenza, ovvero di una media sul caso peggiore: a ogni operazione si attribuisce una parte del costo totale, anche se in quel momento la sua esecuzione è stata più economica o più costosa.

> **Osservazione (Non è un'analisi probabilistica).** La stima ottenuta **non è probabilistica**: non si fanno ipotesi sulla distribuzione delle operazioni e non si calcolano valori attesi. Si considera la sequenza peggiore possibile e si garantisce che, anche in quel caso, il costo medio per operazione resta contenuto, ovvero che il costo ammortizzato è un limite valido per qualsiasi sequenza di operazioni.

L'approccio naive. Un modo naturale per calcolare la complessità di una sequenza di n operazioni consiste nel calcolare il caso peggiore di una *singola* operazione e nel moltiplicarlo per n. Vedremo che questo approccio dà luogo a un'analisi **troppo pessimistica**. Infatti, in molte strutture dati è impossibile che tutte le operazioni di una sequenza ricadano nel caso peggiore: anzi, spesso solo una parte molto limitata di esse può effettivamente incorrere in tale situazione. L'analisi ammortizzata serve proprio a tenere conto di questo fatto.

I tre metodi. Per ottenere stime più strette si usano tre tecniche:

- **Metodo dell'aggregazione**: si stima il costo totale $T(n)$ della sequenza e lo si divide in parti uguali tra le n operazioni (sezione 1.3).
- **Metodo degli accantonamenti** (accounting method): si assegna a ciascuna operazione un costo ammortizzato scelto da noi, in modo che la somma dei costi ammortizzati non sia mai inferiore al costo reale; l'eccedenza viene immagazzinata come *credito* su singoli elementi della struttura dati (sezione 1.4).
- **Metodo del potenziale** (potential method): come il precedente, ma il credito viene associato alla struttura dati nel suo insieme tramite una *funzione potenziale* (sezione 1.5).

1.2 Due esempi di riferimento

Per confrontare i tre metodi useremo sempre gli stessi due esempi, così da vedere come lo stesso problema venga affrontato in modi diversi.

1.2.1 Stack con MultiPop

Uno stack S è una struttura dati che si accede solo dalla cima, e che supporta le seguenti operazioni elementari:

- Push(S, x): inserisce l'elemento x in cima allo stack.
- Pop(S): rimuove l'elemento in cima allo stack, se c'è; su uno stack vuoto non ha alcun effetto.
- StackEmpty(S): restituisce vero se lo stack è vuoto, falso altrimenti.

Ognuna di queste operazioni costa $O(1)$, e per semplicità attribuiamo costo 1 a Push e a Pop. Più precisamente, contiamo soltanto i Push e i Pop *effettivi*, ovvero quelli che inseriscono o rimuovono davvero un elemento: un Pop eseguito su uno stack vuoto ha quindi costo 0. Aggiungiamo ora l'operazione **MultiPop**(S, k), che rimuove i k elementi in cima allo stack, oppure l'intero stack se questo ne contiene meno di k. Si tratta di un'operazione *derivata*, ovvero ottenuta ripetendo più volte un'operazione elementare.

Algoritmo 1 L'operazione MultiPop(S, k) ripete Pop finché lo stack non si svuota o finché non sono stati rimossi k elementi.

```
1: procedure MultiPop(S, k)
2:     while not StackEmpty(S) and k > 0 do
3:         Pop(S)
4:         k ← k − 1
5:     end while
6: end procedure
```

Il costo di MultiPop(S, k) è pari al numero di iterazioni del ciclo, ovvero al numero di Pop effettivamente eseguiti:

$$\text{costo}\big(\textsc{MultiPop}(S, k)\big) = \min(|S|,\ k)$$

dove $|S|$ è il numero di elementi presenti nello stack al momento della chiamata. La figura 1.1 mostra un esempio.

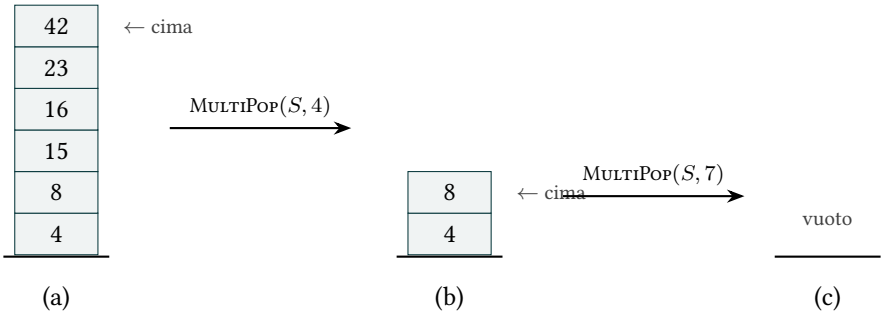


Figura 1.1. Effetto di MultiPop su uno stack contenente sei elementi (a, la cima è l'elemento 42). MultiPop$(S, 4)$ rimuove i quattro elementi in cima e lascia 4 e 8 (b); la chiamata successiva MultiPop$(S, 7)$ svuota lo stack (c), perché ne erano rimasti solo due: in questo caso il costo è $\min(2, 7) = 2$ e non 7.

Analisi naive. Consideriamo una sequenza di n operazioni (Push, Pop e MultiPop) a partire da uno stack inizialmente vuoto. In questa situazione:

- Lo stack contiene al più n elementi, quindi $|S| = O(n)$.
- Una singola operazione costa quindi $O(n)$ nel caso peggiore, perché un MultiPop può dover rimuovere $O(n)$ elementi.
- Il costo di n operazioni è allora $n \cdot O(n) = O(n^2)$.

Questo limite superiore è corretto, ma non è **stretto**[1]: abbiamo ipotizzato che ogni operazione sia un MultiPop costoso, e vedremo che ciò è impossibile.

[1] Ovvero non è anche un limite $\Omega(n^2)$: nessuna sequenza di n operazioni costa davvero quanto n^2.

1.2.2 Contatore binario con INCREMENT

Il secondo esempio è un contatore binario che conta in avanti a partire da 0. Sia $A[0 \dots k-1]$ un array di k bit, in cui $A[0]$ è il bit meno significativo. Il valore memorizzato nel contatore è

$$\text{value}[A] = \sum_{i=0}^{k-1} A[i] \cdot 2^i.$$

L'operazione INCREMENT somma 1 al valore, modulo 2^k:

$$\text{value}[\textsc{Increment}(A)] \equiv \text{value}[A] + 1 \pmod{2^k}.$$

Ad esempio, con $k = 7$ il contatore 1101011 (che vale 107) diventa 1101100 (che vale 108): gli ultimi due bit passano da 1 a 0, il terzo da 0 a 1, e gli altri restano invariati.

Algoritmo 2 L'operazione INCREMENT(A) azzera i bit uguali a 1 a partire dal meno significativo (propagando il riporto) e porta a 1 il primo bit uguale a 0 che incontra.

```
1:  procedure INCREMENT(A)
2:      i ← 0
3:      while i < k and A[i] = 1 do
4:          A[i] ← 0
5:          i ← i + 1
6:      end while
7:      if i < k then
8:          A[i] ← 1
9:      end if
10: end procedure
```

Le operazioni elementari sono la modifica di un singolo bit, e ne distinguiamo due tipi:

- **SET**: porta un bit da 0 a 1.
- **RESET**: porta un bit da 1 a 0.

Ciascuna ha costo 1, quindi il costo di un INCREMENT è il numero di bit che cambiano. Notiamo che ogni INCREMENT esegue *al più un* SET (riga 8 dell'algoritmo 2), e può non eseguirne nessuno solo quando il contatore passa da $1 \dots 1$ a $0 \dots 0$.

Analisi naive. Nel caso peggiore l'array contiene tutti 1: l'incremento azzera tutti i k bit, quindi

$$\text{costo di un incremento} = O(k), \qquad \text{costo di } n \text{ incrementi} = n \cdot O(k) = O(nk).$$

Anche in questo caso si tratta di un'analisi molto sovrabbondante: un incremento modifica tutti i k bit soltanto nel passaggio da $01 \dots 1$ a $10 \dots 0$ e in quello da $1 \dots 1$ a $0 \dots 0$, cioè due volte ogni 2^k incrementi. Subito dopo il contatore ha al più un bit uguale a 1, e gli incrementi successivi costano pochissimo.

1.3 Metodo dell'aggregazione

Il problema delle analisi naive viste sopra è che calcolano il caso peggiore di una singola operazione e poi lo moltiplicano per n. L'idea dell'aggregazione è non prendere questa scorciatoia: si stima direttamente il costo totale della sequenza, tenendo conto di come le operazioni interagiscono tra loro.

Definizione 1.2 (Metodo dell'aggregazione). Il **metodo dell'aggregazione** (aggregate analysis) consiste nello stimare il costo $T(n)$ di una sequenza di n operazioni e nel distribuirlo in modo equo tra le n operazioni, ottenendo un costo ammortizzato

$$\frac{T(n)}{n}$$

per ciascuna operazione, indipendentemente dal suo tipo.

1.3.1 Stack con MultiPop

Partiamo da uno stack inizialmente vuoto. Le operazioni Push e Pop sono le operazioni *elementari*, mentre MultiPop è un'operazione *derivata*: ogni sequenza di operazioni $\langle op_1, \ldots, op_n \rangle$ con $op_i \in \{\textsc{Push}, \textsc{Pop}, \textsc{MultiPop}\}$ può essere **srotolata** in una sequenza equivalente $\langle op'_1, \ldots, op'_m \rangle$ di sole operazioni elementari, con $op'_j \in \{\textsc{Push}, \textsc{Pop}\}$, sostituendo

$$\begin{aligned} \textsc{Pop}(S) &\longrightarrow \textsc{Pop}(S) \\ \textsc{Push}(S, x) &\longrightarrow \textsc{Push}(S, x) \\ \textsc{MultiPop}(S, k) &\longrightarrow \underbrace{\textsc{Pop}(S), \ldots, \textsc{Pop}(S)}_{\min(|S|, k)} \end{aligned}$$

Poiché ogni operazione elementare costa 1, le due sequenze hanno lo stesso costo, ovvero il numero m di operazioni elementari:

$$\text{costo}\big(\langle op_1, \ldots, op_n \rangle\big) = \text{costo}\big(\langle op'_1, \ldots, op'_m \rangle\big) = m.$$

Esempio. Consideriamo la sequenza $\textsc{Push}(a)$, $\textsc{Push}(b)$, $\textsc{Push}(c)$, $\textsc{MultiPop}(4)$, $\textsc{Push}(d)$, Pop, ... e osserviamo come viene srotolata in figura 1.2. Quando viene eseguito $\textsc{MultiPop}(4)$ lo stack contiene soltanto tre elementi, quindi l'operazione si traduce in $\min(3, 4) = 3$ operazioni di Pop.

Un secondo esempio. Prendiamo la sequenza $\textsc{Push}(1)$, $\textsc{Push}(2)$, $\textsc{Push}(3)$, Pop, $\textsc{Push}(4)$, $\textsc{MultiPop}(2)$. Le prime cinque operazioni costano 1 ciascuna, ovvero 4 Push e 1 Pop, mentre il MultiPop rimuove 4 e 2 e costa quindi 2: il costo totale è $4 + 1 + 2 = 7$. Se aggiungiamo un altro $\textsc{MultiPop}(2)$, lo stack contiene solo l'elemento 1, quindi si traduce in un solo Pop e il costo totale sale a 8. Notiamo che un MultiPop costa tanto quanti sono gli elementi che toglie, e che non può togliere più elementi di quelli inseriti.

Analisi. Questa osservazione è il punto chiave: ogni elemento può essere rimosso al più una volta per ogni volta che è stato inserito. Nella sequenza srotolata, quindi, il numero di Pop non supera il numero di Push, e il numero di Push della sequenza srotolata coincide con quello della

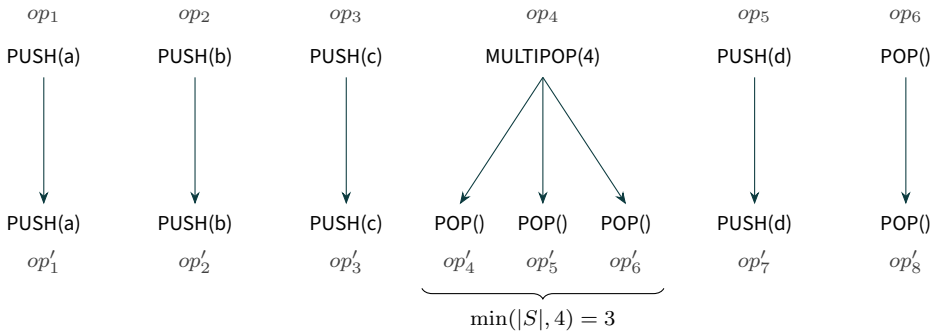


Figura 1.2. Una sequenza di operazioni sullo stack (in alto) e la corrispondente sequenza di sole operazioni elementari (in basso): MultiPop(4), eseguito su uno stack che contiene tre elementi, si srotola in tre Pop (parentesi graffa). La sequenza originale ha 6 operazioni e quella srotolata ne ha 8; ognuna delle 8 costa 1.

sequenza originale, a sua volta al più n:

$$\#\textsc{Pop}\big(\langle op'_1, \ldots, op'_m\rangle\big) \le \#\textsc{Push}\big(\langle op'_1, \ldots, op'_m\rangle\big) = \#\textsc{Push}\big(\langle op_1, \ldots, op_n\rangle\big) \le n.$$

Detta $\langle op_1, \ldots, op_n\rangle$ la sequenza più costosa, si ottiene

$$T(n) = \text{costo}\big(\langle op'_1, \ldots, op'_m\rangle\big) = \#\textsc{Pop} + \#\textsc{Push} \le n + n = 2n$$

e quindi

$$\text{costo ammortizzato per operazione} = \frac{T(n)}{n} \le \frac{2n}{n} = 2 = O(1).$$

Lo stesso costo ammortizzato vale per Push, Pop e MultiPop, anche se un singolo MultiPop può costare molto: è il totale della sequenza che resta limitato da $2n$, non la singola operazione. Rispetto all'analisi naive, che dava $O(n^2)$, il costo di n operazioni scende a $O(n)$.

Esercizio 1.1 (Sequenza più costosa sullo stack). Dimostrare che la sequenza più costosa di n operazioni Push, Pop e MultiPop, a partire da uno stack vuoto, ha costo

$$T(n) \le \begin{cases} 1 & \text{se } n = 1 \\ 2(n-1) & \text{se } n \ge 2. \end{cases}$$

Soluzione. Se $n = 1$ le sequenze possibili sono tre: $\langle$Push$\rangle$, di costo 1, $\langle$Pop$\rangle$, di costo 0 perché lo stack è vuoto, e $\langle$MultiPop$(k)\rangle$ con $k = 1, 2, \ldots$, anch'esse di costo 0. Quindi $T(1) \le 1$.

Sia ora $n \ge 2$. Indichiamo con p il numero di Push della sequenza e con r il numero di elementi rimossi da Pop e MultiPop. Ogni elemento può essere rimosso solo dopo essere stato inserito, quindi $r \le p$ e

$$T(n) = p + r \le 2p.$$

Distinguiamo due casi:

- Se $r \ge 1$, almeno una delle operazioni è un Pop o un MultiPop, quindi $p \le n - 1$ e $T(n) \le 2(n-1)$.

- Se $r = 0$, il costo è dato dai soli PUSH, e poiché $n - 2 \geq 0$ si ha $T(n) = p \leq n \leq n + (n-2) = 2(n-1)$.

Il limite $2(n-1)$ si raggiunge davvero: basta eseguire $n-1$ operazioni PUSH seguite da MULTIPOP$(S, n-1)$, che costano $(n-1) + (n-1) = 2(n-1)$.

1.3.2 Contatore binario con INCREMENT

Supponiamo ora che il contatore sia inizialmente nullo, e studiamo quanti bit cambiano in una sequenza di n incrementi. Il punto è che non tutti i bit cambiano a ogni chiamata, come si vede dalla figura 1.3:

- $A[0]$ cambia a ogni incremento, ovvero n volte.
- $A[1]$ cambia solo una volta ogni due incrementi, ovvero $\lfloor n/2 \rfloor$ volte.
- $A[2]$ cambia solo una volta ogni quattro incrementi, ovvero $\lfloor n/2^2 \rfloor$ volte.

In generale, per $i = 0, 1, \ldots, k-1$ il bit $A[i]$ cambia $\lfloor n/2^i \rfloor$ volte nell'arco di n incrementi[2]; per $i \geq k$ il bit non esiste e non può cambiare.

valore	$A[7]$	$A[6]$	$A[5]$	$A[4]$	$A[3]$	$A[2]$	$A[1]$	$A[0]$	costo totale
0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	1	1
2	0	0	0	0	0	0	1	0	3
3	0	0	0	0	0	0	1	1	4
4	0	0	0	0	0	1	0	0	7
5	0	0	0	0	0	1	0	1	8
6	0	0	0	0	0	1	1	0	10
7	0	0	0	0	0	1	1	1	11
8	0	0	0	0	1	0	0	0	15
9	0	0	0	0	1	0	0	1	16
10	0	0	0	0	1	0	1	0	18
11	0	0	0	0	1	0	1	1	19
12	0	0	0	0	1	1	0	0	22
13	0	0	0	0	1	1	0	1	23
14	0	0	0	0	1	1	1	0	25
15	0	0	0	0	1	1	1	1	26
16	0	0	0	1	0	0	0	0	31

Figura 1.3. Un contatore a 8 bit durante 16 incrementi consecutivi, a partire da 0. A sinistra il valore del contatore, al centro i bit $A[7], \ldots, A[0]$ (i bit evidenziati sono quelli che sono cambiati nel passaggio dal valore precedente), a destra il costo totale, ovvero il numero complessivo di bit modificati dall'inizio. $A[0]$ cambia a ogni incremento, $A[1]$ una volta ogni due, $A[2]$ una volta ogni quattro e così via; il costo totale è sempre minore del doppio del numero di incrementi.

[2] Il bit $A[i]$ cambia nel passaggio dal valore v al valore $v+1$ se e solo se $v+1$ è un multiplo di 2^i. Partendo da 0, i valori $v+1$ percorsi sono $1, \ldots, n$, tra i quali i multipli di 2^i sono esattamente $\lfloor n/2^i \rfloor$.

Il costo totale è il numero complessivo di bit che cambiano, quindi

$$T(n) = \sum_{i=0}^{k-1} \left\lfloor \frac{n}{2^i} \right\rfloor \leq \sum_{i=0}^{k-1} \frac{n}{2^i} < \sum_{i=0}^{\infty} \frac{n}{2^i} = n \sum_{i=0}^{\infty} \frac{1}{2^i} = 2n$$

dove abbiamo usato il fatto che la serie geometrica $\sum_{i \geq 0} 2^{-i}$ vale 2. Ne segue che

$$\text{costo ammortizzato per operazione} = \frac{T(n)}{n} < \frac{2n}{n} = 2 = O(1)$$

contro il $O(k)$ dell'analisi naive. In figura, ad esempio, 16 incrementi costano $31 < 32 = 2 \cdot 16$.

> **Esercizio 1.2 (Costo di una sequenza di incrementi).** Dimostrare che la sequenza di n incrementi su un contatore binario a k bit, inizialmente nullo, ha costo $T(n)$ tale che
>
> $$n \leq T(n) \leq 2n - 1$$
>
> dove le due disuguaglianze non sono in generale rafforzabili.

Soluzione. Ogni incremento modifica almeno un bit, quindi $T(n) \geq n$. Per il limite superiore basta il calcolo appena fatto con l'aggregazione, che dà $T(n) < 2n$: poiché $T(n)$ è un numero intero, ne segue

$$T(n) \leq 2n - 1.$$

Le due disuguaglianze non si possono migliorare. Con $k = 1$ ogni incremento modifica esattamente un bit, quindi $T(n) = n$. Per la seconda consideriamo $n = 2^i$ con $1 \leq i \leq k - 1$: i bit $A[j]$ con $j > i$ non cambiano mai, perché $\lfloor 2^i / 2^j \rfloor = 0$, e quindi

$$T(2^i) = 2^i + \left\lfloor \frac{2^i}{2} \right\rfloor + \cdots + \left\lfloor \frac{2^i}{2^{i-1}} \right\rfloor + \left\lfloor \frac{2^i}{2^i} \right\rfloor = 2^i + 2^{i-1} + \cdots + 2 + 1 = 2^{i+1} - 1 = 2n - 1.$$

Ad esempio $T(4) = 1 + 2 + 1 + 3 = 7 = 2 \cdot 4 - 1$.

1.3.3 Contatore non necessariamente nullo

Se il contatore parte da una configurazione qualsiasi, il bit $A[i]$ continua a cambiare una volta ogni 2^i incrementi, ma il primo cambiamento può arrivare subito. Per la stessa ragione della nota precedente, il numero di volte in cui $A[i]$ cambia è il numero di multipli di 2^i tra n interi consecutivi, che è al più $\lceil n/2^i \rceil$. Si ottiene quindi

$$T(n) \leq \sum_{i=0}^{k-1} \left\lceil \frac{n}{2^i} \right\rceil \leq \sum_{i=0}^{k-1} \left(\frac{n}{2^i} + 1 \right) = n \sum_{i=0}^{k-1} \frac{1}{2^i} + k < 2n + k.$$

Il costo di n incrementi è quindi $O(n + k)$: se $n = \Omega(k)$, ovvero se si eseguono almeno tanti incrementi quanti sono i bit, il costo è $O(n)$ qualunque sia la configurazione di partenza, e il costo ammortizzato resta $O(1)$.

Esempio. Partiamo dal contatore a 6 bit 011011 ed eseguiamo $n = 5$ incrementi, che producono nell'ordine 011100, 011101, 011110, 011111 e 100000. Il bit $A[0]$ cambia 5 volte, $A[1]$ cambia 3 volte (nel primo, nel terzo e nel quinto incremento), $A[2]$ due volte, mentre $A[3]$, $A[4]$ e $A[5]$ cambiano una volta soltanto, nell'ultimo incremento. Questi numeri coincidono con i limiti $\lceil 5/2^i \rceil = 5, 3, 2, 1, 1, 1$, e il costo totale è $13 < 2n + k = 16$.

Esercizio 1.3 (MultiPush). Se l'insieme delle operazioni sullo stack includesse un'operazione MultiPush$(S, x_1, \ldots, x_k)$, che inserisce k elementi nello stack, il limite $O(1)$ sul costo ammortizzato delle operazioni sullo stack sarebbe ancora valido?

Soluzione. No, in generale. Un MultiPush costa k e può inserire fino a k elementi, quindi una sequenza di n operazioni MultiPush su uno stack vuoto costa nk e ha costo ammortizzato k, che non è $O(1)$ se k non è limitato da una costante. L'argomento del paragrafo precedente cade perché non è più vero che il numero di Push è al più n: la sequenza ne esegue anche nk.

Esercizio 1.4 (Decrement). Si dimostri che, se un'operazione Decrement fosse inclusa nell'esempio del contatore a k bit, n operazioni potrebbero richiedere tempo $\Theta(nk)$.

Soluzione. Ogni operazione modifica al più k bit, quindi n operazioni costano $O(nk)$. Per il limite inferiore partiamo da $0 \ldots 0$ ed eseguiamo alternativamente Decrement e Increment: il primo porta il contatore a $1 \ldots 1$ (modulo 2^k) modificando k bit, il secondo lo riporta a $0 \ldots 0$ modificandone ancora k. Ogni operazione costa quindi k e il totale è $nk = \Theta(nk)$. L'analisi dell'incremento non si estende perché sfrutta il fatto che un bit può essere azzerato solo dopo essere stato portato a 1; con Decrement il contatore può tornare indietro e il caso peggiore si ripete a ogni passo.

Esercizio 1.5 (Costo pari a una potenza di 2). Una sequenza di n operazioni viene eseguita su una struttura dati. L'i-esima operazione costa i se i è una potenza esatta di 2, altrimenti costa 1. Applicare il metodo dell'aggregazione per determinare il costo ammortizzato per operazione. Lo stesso esercizio verrà risolto con il metodo degli accantonamenti (sezione 1.4) e con quello del potenziale (sezione 1.5).

Soluzione. Indichiamo con c_i il costo dell'i-esima operazione, quindi $c_i = i$ se $i = 2^j$ per un certo $j \in \mathbb{N}$ e $c_i = 1$ altrimenti. Separiamo nella somma le operazioni che costano 1 da quelle che costano i:

$$T(n) = \sum_{i=1}^{n} c_i = \sum_{\substack{i=1 \\ i \notin 2^{\mathbb{N}}}}^{n} 1 + \sum_{\substack{i=1 \\ i \in 2^{\mathbb{N}}}}^{n} i.$$

Tra 1 e n ci sono $\lfloor \lg n \rfloor + 1$ potenze di 2, ovvero $2^0, 2^1, \ldots, 2^{\lfloor \lg n \rfloor}$[3], quindi la prima somma vale $n - (\lfloor \lg n \rfloor + 1)$, mentre la seconda è una somma geometrica:

$$\sum_{\substack{i=1 \\ i \in 2^{\mathbb{N}}}}^{n} i = \sum_{j=0}^{\lfloor \lg n \rfloor} 2^j = 2^{\lfloor \lg n \rfloor + 1} - 1.$$

Si ottiene allora

$$T(n) = n - (\lfloor \lg n \rfloor + 1) + 2^{\lfloor \lg n \rfloor + 1} - 1 < n + 2^{\lfloor \lg n \rfloor + 1} \leq n + 2n = 3n$$

dove abbiamo usato $2^{\lfloor \lg n \rfloor} \leq n$. Pertanto

$$\text{costo ammortizzato per operazione} = \frac{T(n)}{n} < \frac{3n}{n} = 3.$$

1.4 Metodo degli accantonamenti

Il metodo dell'aggregazione assegna lo stesso costo ammortizzato a tutte le operazioni, e questo può essere restrittivo quando le operazioni sono di natura diversa. Il metodo degli accantonamenti (accounting method) permette invece di assegnare a ciascuna operazione un costo ammortizzato scelto da noi, anche diverso da operazione a operazione.

Definizione 1.3 (Costi ammortizzati). Data una sequenza di operazioni $op_1, op_2, \ldots, op_n$, indichiamo con

$$c_i = \text{costo reale}(op_i), \qquad \hat{c}_i = \text{costo ammortizzato}(op_i)$$

dove i costi ammortizzati $\hat{c}_i$ sono *definiti da noi*. L'**obiettivo** è definirli in modo tale che, per qualsiasi sequenza di operazioni, valga

$$T(n) = \sum_{i=1}^{n} c_i \leq \sum_{i=1}^{n} \hat{c}_i.$$

In questo modo la somma dei costi ammortizzati è un limite superiore al costo reale, e basta stimare la prima per avere una stima della seconda.

L'idea del credito. Per garantire la disuguaglianza si ragiona come in un conto in banca, usando il costo ammortizzato come "prezzo" addebitato a ogni operazione:

- Se $\hat{c}_i > c_i$, delle $\hat{c}_i$ unità di costo addebitate, c_i servono a pagare il costo reale di op_i, mentre le restanti $\hat{c}_i - c_i$ vengono **immagazzinate** su elementi specifici della struttura dati, come credito.
- Se $c_i > \hat{c}_i$, la differenza $c_i - \hat{c}_i$ viene **recuperata** dai crediti immagazzinati nella struttura dati.

[3] Con lg si indica il logaritmo in base 2.

Perché il metodo funzioni, il credito deve essere sufficiente a coprire ogni operazione "in perdita". Per formalizzare questa idea, sia D_0 la configurazione iniziale di una struttura dati, sia $op_1, \dots, op_n$ una sequenza di operazioni e sia D_j la configurazione che si ottiene da D_{j-1} applicando op_j, ovvero

$$D_0 \xrightarrow{op_1} D_1 \xrightarrow{op_2} D_2 \xrightarrow{op_3} \cdots \xrightarrow{op_n} D_n.$$

Indichiamo con B_j il **credito totale** accumulato dopo l'esecuzione delle prime j operazioni, ovvero la differenza tra quanto è stato addebitato e quanto è stato effettivamente speso:

$$B_j = \sum_{i=1}^{j} (\hat{c}_i - c_i), \qquad B_0 = 0.$$

Lemma 1.1 (Correttezza degli accantonamenti). Se i costi ammortizzati sono scelti in modo che

$$B_j \geq 0 \qquad \text{per } j = 0, 1, \dots, n$$

allora $\sum_{i=1}^{j} c_i \leq \sum_{i=1}^{j} \hat{c}_i$ per ogni j, e in particolare $\sum_{i=1}^{n} c_i \leq \sum_{i=1}^{n} \hat{c}_i$.

Dimostrazione. Per ogni $j = 1, \dots, n$ si ha, per ipotesi,

$$0 \leq B_j = \sum_{i=1}^{j} (\hat{c}_i - c_i) = \sum_{i=1}^{j} \hat{c}_i - \sum_{i=1}^{j} c_i, \qquad \text{quindi} \qquad \sum_{i=1}^{j} c_i \leq \sum_{i=1}^{j} \hat{c}_i.$$

Scegliendo $j = n$ si ottiene la disuguaglianza sull'intera sequenza. □

In pratica, per applicare il metodo basta scegliere i costi $\hat{c}_i$ e verificare che il credito totale B_j non diventi mai negativo: in questo modo la somma dei costi ammortizzati è una stima per eccesso del costo effettivo della sequenza.

Osservazione (I crediti esistono solo nell'analisi). Il credito è uno strumento di analisi: non compare nel codice e non occorre memorizzarlo da nessuna parte (ad esempio, non serve aggiungere un campo $x.\mathrm{credit}$ a ogni elemento). Serve solo a convincerci che i costi $\hat{c}_i$ scelti sono sufficienti.

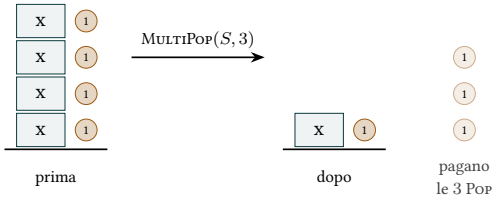
1.4.1 Stack con MultiPop

Assumiamo di partire da uno stack vuoto e scegliamo i costi ammortizzati

$$\hat{c}_{\text{Push}} = 2, \qquad \hat{c}_{\text{Pop}} = \hat{c}_{\text{MultiPop}} = 0$$

a fronte dei costi reali $c_{\text{Push}} = 1$, $c_{\text{Pop}} = 1$ e $c_{\text{MultiPop}} = \min(|S|, k)$. Questi costi hanno una lettura molto intuitiva: un Push addebita 2 unità, di cui 1 paga il costo reale dell'operazione e l'altra viene **assegnata all'elemento** appena inserito, che porta quindi con sé una unità di credito. Quando l'elemento viene rimosso, con un Pop o con un MultiPop, il costo reale della rimozione si paga proprio con quell'unità, senza addebitare nulla all'operazione. Il credito è quindi $|S|$, cioè una unità per ogni elemento presente nello stack.

Esempio. Consideriamo uno stack con quattro elementi, ognuno dei quali porta con sé una moneta, cioè l'unità di credito accantonata dal suo PUSH. Eseguiamo MULTIPOP$(S, 3)$: il costo reale è 3, ma al MULTIPOP non addebitiamo nulla. Le tre monete degli elementi rimossi bastano a pagare le tre operazioni POP, e sullo stack resta un solo elemento con la sua moneta: il credito è sempre uguale al numero di elementi presenti.



Verifichiamo che il credito non sia mai negativo. In ogni istante j vale

$$B_j = \sum_{i=1}^{j} \hat{c}_i - \sum_{i=1}^{j} c_i = |S_j| \geq 0$$

dove $|S_j|$ è la dimensione dello stack dopo j operazioni: un PUSH aggiunge $2 - 1 = 1$ al primo membro e alla dimensione, mentre un POP o un MULTIPOP che rimuovono k' elementi aggiungono $0 - k' = -k'$ al primo membro e tolgono k' alla dimensione. Per il lemma 1.1 segue

$$\sum_{i=1}^{n} c_i \leq \sum_{i=1}^{n} \hat{c}_i = 2 \cdot \#\textsc{Push} \leq 2n$$

e quindi, come con l'aggregazione, il costo di n operazioni è $O(n)$. Si noti che questa volta il costo ammortizzato *non* è lo stesso per tutte le operazioni: vale 2 per PUSH e 0 per le altre due, ma questo è proprio ciò che rende il metodo più flessibile.

1.4.2 Contatore binario con INCREMENT

Consideriamo il contatore inizialmente nullo e usiamo come costo reale il numero di bit modificati. Come costi ammortizzati delle sottooperazioni scegliamo

$$\hat{c}_{\textsc{Set}} = 2, \qquad \hat{c}_{\textsc{Reset}} = 0.$$

Un SET addebita quindi 2 unità: 1 serve a pagare l'operazione e l'altra viene accantonata **sul bit stesso**. Poiché ogni bit uguale a 1 è stato portato a 1 da un SET, ogni bit uguale a 1 porta con sé una unità di credito, che usiamo per pagare il RESET quando lo riportiamo a 0. Un INCREMENT esegue al più un SET (e un numero qualsiasi di RESET, che non vengono addebitati), quindi

$$\hat{c}_{\textsc{Increment}} = 2 \cdot \#\textsc{Set} = \begin{cases} 0 & \text{se non esegue alcun SET} \\ 2 & \text{se ne esegue uno} \end{cases} \leq 2.$$

Ad esempio, nel passaggio da 0111 a 1000 i tre bit azzerati usano le loro tre unità di credito, mentre il bit portato a 1 paga una unità e ne conserva un'altra.

Il credito immagazzinato in ogni istante è il numero di bit uguali a 1 presenti nel contatore,

quindi non è mai negativo:

$$B_j = \sum_{i=1}^{j} \hat{c}_i - \sum_{i=1}^{j} c_i = \#\{\text{bit del contatore uguali a } 1\} \geq 0.$$

Per il lemma 1.1 vale allora

$$\sum_{i=1}^{n} c_i \leq \sum_{i=1}^{n} \hat{c}_i \leq 2n$$

cioè il costo di n incrementi è $O(n)$ e il costo ammortizzato di un incremento è $O(1)$, ritrovando il risultato dell'aggregazione.

Esercizio 1.6 (Costo pari a una potenza di 2, con gli accantonamenti). Risolvere l'esercizio 1.5 con il metodo degli accantonamenti.

Soluzione. Scegliamo i costi ammortizzati

$$\hat{c}_1 = 1, \qquad \hat{c}_i = 2 \text{ se } i = 2^j \text{ con } j \geq 1, \qquad \hat{c}_i = 3 \text{ altrimenti.}$$

Un'operazione che costa 1 (ovvero con i non potenza di 2) paga 1 per sé e accantona 2 unità di credito. Un'operazione $i = 2^j$ con $j \geq 1$ costa 2^j, ma tra l'operazione 2^{j-1} e l'operazione 2^j ci sono $2^{j-1} - 1$ operazioni da 1, che hanno accantonato complessivamente $2(2^{j-1} - 1) = 2^j - 2$ unità: queste pagano quasi tutto il costo 2^j e le 2 unità addebitate coprono la parte restante. In questo modo il credito accumulato dopo i operazioni è

$$B_i = 2\left(i - 2^{\lfloor \lg i \rfloor}\right) \geq 0$$

(ad esempio 2 dopo l'operazione 3, 0 dopo la 4, 6 dopo la 7 e di nuovo 0 dopo la 8) e non è mai negativo. Per il lemma 1.1

$$T(n) \leq \sum_{i=1}^{n} \hat{c}_i \leq 3n$$

e il costo ammortizzato per operazione è al più 3.

1.5 Metodo del potenziale

Il metodo degli accantonamenti associa il credito a singoli elementi della struttura dati. Il metodo del potenziale (potential method) segue la stessa logica, ma immagazzina il credito nella struttura dati *nel suo insieme*, come se fosse un'energia potenziale che può essere rilasciata per pagare le operazioni future.

Definizione 1.4 (Funzione potenziale). Una **funzione potenziale** associa a ogni configurazione D della struttura dati un numero reale

$$D \longmapsto \Phi(D) \in \mathbb{R}$$

detto **potenziale** di D. Data la sequenza di configurazioni $D_0 \xrightarrow{op_1} D_1 \xrightarrow{op_2} \cdots \xrightarrow{op_n} D_n$ e detto c_i il costo reale di op_i, il **costo ammortizzato** di op_i rispetto a Φ è

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) = c_i + \Delta\Phi_i$$

dove $\Delta\Phi_i = \Phi(D_i) - \Phi(D_{i-1})$ è la **variazione di potenziale** causata da op_i.

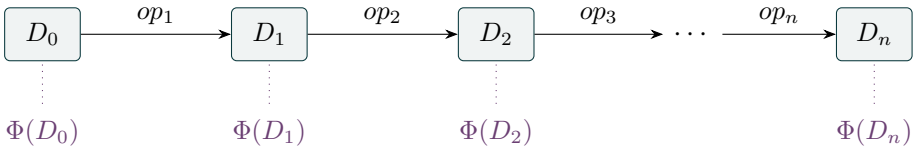


Figura 1.4. Il metodo del potenziale associa a ogni configurazione D_i della struttura dati un valore $\Phi(D_i)$: il costo ammortizzato dell'operazione op_i è il suo costo reale più la differenza tra il potenziale dopo e prima dell'operazione.

Intuitivamente, se $\Delta\Phi_i > 0$ l'operazione viene sovrapagata e il potenziale della struttura dati aumenta, mentre se $\Delta\Phi_i < 0$ l'operazione viene sottopagata e la diminuzione del potenziale paga la parte di costo reale che non è coperta dal costo ammortizzato. Sommando i costi ammortizzati, i termini di potenziale si semplificano a coppie:

$$\begin{aligned}\sum_{i=1}^{n} \hat{c}_i &= \sum_{i=1}^{n} \big[c_i + \Phi(D_i) - \Phi(D_{i-1})\big] \\ &= \big(c_1 + \Phi(D_1) - \Phi(D_0)\big) + \big(c_2 + \Phi(D_2) - \Phi(D_1)\big) + \cdots + \big(c_n + \Phi(D_n) - \Phi(D_{n-1})\big)\end{aligned}$$

dove ogni $\Phi(D_i)$, con $0 < i < n$, compare una volta con segno positivo e una con segno negativo (somma telescopica), da cui

$$\sum_{i=1}^{n} \hat{c}_i = \sum_{i=1}^{n} c_i + \Phi(D_n) - \Phi(D_0). \tag{1.1}$$

Lemma 1.2 (Correttezza del potenziale). Vale $\sum_{i=1}^{n} \hat{c}_i \geq \sum_{i=1}^{n} c_i$ se e solo se $\Phi(D_n) \geq \Phi(D_0)$.

Dimostrazione. Per la (1.1), la disuguaglianza $\sum \hat{c}_i \geq \sum c_i$ equivale a $\sum c_i + \Phi(D_n) - \Phi(D_0) \geq \sum c_i$, ovvero a $\Phi(D_n) - \Phi(D_0) \geq 0$. □

Nella pratica non si sa in anticipo quante operazioni verranno eseguite, quindi si richiede la condizione per ogni k:

$$\Phi(D_k) \geq \Phi(D_0) \qquad \text{per } k = 1, 2, \ldots, n.$$

Di solito si definisce $\Phi(D_0) = 0$ e si dimostra che $\Phi(D_k) \geq 0$ per ogni k. Notiamo che i costi ammortizzati dipendono dalla funzione potenziale scelta: funzioni diverse danno costi diversi,

tutti però validi come limiti superiori al costo reale. La scelta migliore dipende dai limiti che si vogliono dimostrare.

1.5.1 Stack con MultiPop

Partendo da uno stack inizialmente vuoto, definiamo il potenziale come il numero di elementi dello stack:

$$\Phi(S) = |S|.$$

Se S_0 è lo stack vuoto si ha $\Phi(S_0) = 0$, e poiché la dimensione di uno stack non è mai negativa vale $\Phi(S) \geq \Phi(S_0)$ per ogni configurazione. Calcoliamo i costi ammortizzati delle tre operazioni, indicando con S_{i-1} e S_i lo stack prima e dopo l'operazione:

$$\begin{aligned}
\hat{c}_{\text{Push}} &= c_{\text{Push}} + |S_i| - |S_{i-1}| = 1 + 1 = 2 && \text{perché } |S_i| = |S_{i-1}| + 1,\\
\hat{c}_{\text{Pop}} &= c_{\text{Pop}} + |S_i| - |S_{i-1}| = 1 - 1 = 0 && \text{perché } |S_i| = |S_{i-1}| - 1,\\
\hat{c}_{\text{MultiPop}} &= c_{\text{MultiPop}} + |S_i| - |S_{i-1}| = k' - k' = 0 && \text{perché } |S_i| = |S_{i-1}| - k',
\end{aligned}$$

dove $k' = \min(|S_{i-1}|, k)$ è il numero di elementi effettivamente rimossi. Troviamo gli stessi costi del metodo degli accantonamenti (il potenziale $|S|$ è la somma dei crediti accantonati sugli elementi), e per il lemma 1.2

$$\sum_{i=1}^{n} c_i \leq \sum_{i=1}^{n} \hat{c}_i = 2 \cdot \#\text{Push} \leq 2n.$$

1.5.2 Contatore binario con Increment

Per il contatore, con D_i configurazione dopo i incrementi, definiamo il potenziale come il numero di bit uguali a 1:

$$\Phi(D_i) = \#\{\text{bit uguali a 1 in } D_i\}.$$

Se D_0 è la configurazione nulla si ha $\Phi(D_0) = 0$ e in generale $\Phi(D_i) \geq \Phi(D_0)$. Per calcolare il costo ammortizzato dell'i-esimo incremento osserviamo che

$$c_i = \#\text{Set}_i + \#\text{Reset}_i, \qquad \Delta\Phi_i = \#\text{Set}_i - \#\text{Reset}_i$$

dove $\#\text{Set}_i$ e $\#\text{Reset}_i$ sono il numero di bit portati a 1 e a 0 dall'i-esimo incremento, con $0 \leq \#\text{Set}_i \leq 1$. Infatti ogni Set aggiunge un bit uguale a 1 e ogni Reset ne toglie uno. Pertanto

$$\hat{c}_i = c_i + \Delta\Phi_i = \big(\#\text{Set}_i + \#\text{Reset}_i\big) + \big(\#\text{Set}_i - \#\text{Reset}_i\big) = 2 \cdot \#\text{Set}_i \leq 2$$

da cui

$$\sum_{i=1}^{n} c_i \leq \sum_{i=1}^{n} \hat{c}_i = 2 \cdot \sum_{i=1}^{n} \#\text{Set}_i \leq 2n.$$

1.5.3 Configurazione iniziale non nulla

Il vantaggio del metodo del potenziale è che gestisce con facilità il caso in cui la struttura dati *non* parta dalla configurazione iniziale "vuota". Dalla (1.1) si ottiene infatti

$$\sum_{i=1}^{n} c_i = \sum_{i=1}^{n} \hat{c}_i - \Phi(D_n) + \Phi(D_0)$$

e basta maggiorare il potenziale iniziale.

Contatore non nullo. Usiamo la stessa funzione potenziale di prima. Si ha $\Phi(D_0) \leq k$, dove k è il numero di bit del contatore, ma in generale non vale più $\Phi(D_i) \geq \Phi(D_0)$ (il contatore può passare per configurazioni con meno bit uguali a 1 di quelli iniziali). Il lemma 1.2 non è quindi applicabile, ma la (1.1) dà comunque, poiché $\Phi(D_n) \geq 0$:

$$\sum_{i=1}^{n} c_i = \sum_{i=1}^{n} \hat{c}_i - \Phi(D_n) + \Phi(D_0) \leq 2n + \Phi(D_0) \leq 2n + k.$$

Se $k = O(n)$ allora $2n + k = O(n)$ e quindi $\sum c_i = O(n)$; in generale il costo è $O(n + k)$, come già trovato con l'aggregazione.

Stack non vuoto con MultiPop. Con $\Phi(S) = |S|$ e uno stack iniziale D_0 che contiene $|D_0|$ elementi si ha $\Phi(D_0) = |D_0|$ e $\Phi(D_n) \geq 0$, quindi

$$\sum_{i=1}^{n} c_i = \sum_{i=1}^{n} \hat{c}_i - \Phi(D_n) + \Phi(D_0) \leq 2 \cdot \#\textsc{Push} + |D_0| \leq 2n + |D_0|.$$

Se $|D_0| = O(n)$, ovvero se lo stack iniziale non è troppo grande rispetto al numero di operazioni, allora $\sum c_i = O(n)$.

> **Esercizio 1.7 (Costo pari a una potenza di 2, con il potenziale).** Risolvere l'esercizio 1.5 con il metodo del potenziale.

Soluzione. Poniamo $\Phi(D_0) = 0$ e, per $i \geq 1$,

$$\Phi(D_i) = 2\left(i - 2^{\lfloor \lg i \rfloor}\right)$$

che coincide con il credito B_i accumulato dopo i operazioni nella soluzione con gli accantonamenti, ed è sempre non negativa perché $2^{\lfloor \lg i \rfloor} \leq i$. Calcoliamo i costi ammortizzati distinguendo i casi:

- Se $i = 1$: $\hat{c}_1 = 1 + \Phi(D_1) - \Phi(D_0) = 1 + 0 - 0 = 1$.
- Se i non è una potenza di 2: $c_i = 1$ e $\lfloor \lg i \rfloor = \lfloor \lg(i-1) \rfloor$, quindi $\Phi(D_i) - \Phi(D_{i-1}) = 2$ e $\hat{c}_i = 1 + 2 = 3$.
- Se $i = 2^j$ con $j \geq 1$: $c_i = 2^j$, $\Phi(D_i) = 2(2^j - 2^j) = 0$ e $\Phi(D_{i-1}) = 2\left(2^j - 1 - 2^{j-1}\right) = 2^j - 2$, quindi $\hat{c}_i = 2^j + 0 - (2^j - 2) = 2$.

Si ritrovano gli stessi costi ammortizzati degli accantonamenti, e poiché $\Phi(D_n) \geq 0 = \Phi(D_0)$ vale $T(n) \leq \sum \hat{c}_i \leq 3n$: il costo ammortizzato per operazione è al più 3.

1.6 Confronto tra i metodi

I tre metodi portano agli stessi risultati sugli esempi visti: $O(n)$ per n operazioni su uno stack con MultiPop e per n incrementi di un contatore, quindi $O(1)$ ammortizzato per operazione. Cambia invece il modo di arrivarci, riassunto nella tabella seguente.

	Aggregazione	**Accantonamenti**	**Potenziale**
Costo ammortizzato	Uguale per tutte le operazioni: $T(n)/n$.	Scelto per ogni tipo di operazione.	$\hat{c}_i = c_i + \Delta\Phi_i$, può variare con l'operazione.
Cosa si sceglie	Niente: si stima direttamente $T(n)$.	I costi $\hat{c}_i$.	La funzione Φ.
Dove sta il lavoro pagato in anticipo	Non viene tracciato.	Credito su singoli elementi.	Potenziale della struttura nel suo insieme.
Cosa si verifica	Il calcolo di $T(n)$.	$B_j \geq 0$ per ogni j.	$\Phi(D_k) \geq \Phi(D_0)$ per ogni k.

In sintesi, l'aggregazione è il metodo più semplice ma assegna lo stesso costo a tutte le operazioni, quindi funziona bene quando basta una stima del totale; gli accantonamenti permettono di distinguere le operazioni e hanno una lettura intuitiva in termini di monete, al prezzo di dover decidere dove si trova ogni unità di credito; il potenziale è il più generale, perché riassume tutto in una sola funzione della configurazione e gestisce con facilità il caso di una struttura dati che non parte da zero, ma richiede di trovare la funzione Φ giusta.

1.7 Tabelle dinamiche

Molto spesso non si sa in anticipo quanti elementi dovrà contenere una tabella. Se si alloca poco spazio, prima o poi la tabella si riempie e un nuovo inserimento provoca un overflow; se se ne alloca troppo, gran parte della memoria resta inutilizzata. Le **tabelle dinamiche** risolvono il problema con la **riallocazione**: quando la tabella è piena se ne alloca una più grande, si copiano al suo interno tutti gli elementi e si libera quella vecchia; allo stesso modo, quando la tabella si svuota troppo, la si può sostituire con una più piccola. Una singola riallocazione è costosa, perché richiede di copiare tutti gli elementi, ma avviene raramente: è quindi la situazione ideale per l'analisi ammortizzata, che ci permetterà di dimostrare che inserimenti e cancellazioni hanno costo ammortizzato $O(1)$. Il modo in cui gli elementi sono organizzati all'interno della tabella (un array, uno heap, una tabella hash) non è importante ai fini dell'analisi.

Definizione 1.5 (Fattore di carico). Data una tabella T, indichiamo con $size[T]$ la sua dimensione, ovvero il numero di posizioni disponibili, e con $num[T]$ il numero di elementi che

contiene. Il **fattore di carico** (load factor) di T è

$$\alpha(T) = \begin{cases} \dfrac{num[T]}{size[T]} & \text{se } size[T] > 0 \\ 1 & \text{altrimenti.} \end{cases}$$

Il fattore di carico misura quanto è piena la tabella: vale 1 quando la tabella è piena e diminuisce man mano che si svuota. Per convenzione una tabella vuota, che ha dimensione 0, è considerata piena. Se $\alpha(T)$ resta sempre al di sopra di una costante, lo spazio inutilizzato non supera mai una frazione costante dello spazio totale: è questa la garanzia che vogliamo ottenere, insieme a un costo ammortizzato costante per le operazioni.

1.7.1 Inserimenti con espansione

Consideriamo per ora soltanto gli inserimenti. Quando si inserisce un elemento in una tabella piena, la strategia più comune consiste nel **raddoppiare** la dimensione della tabella: si alloca una nuova tabella di dimensione $2 \cdot size[T]$, vi si copiano tutti gli elementi e si libera quella vecchia. Questa operazione è detta **espansione**, ed è descritta dall'algoritmo 3.

Algoritmo 3 L'operazione TABLE-INSERT(T, x) alloca la tabella al primo inserimento e la raddoppia (espansione) quando è piena, copiando tutti gli elementi nella nuova tabella prima di inserire x.

```
1:  procedure TABLE-INSERT(T, x)
2:      if size[T] = 0 then
3:          allocare Table[T] di dimensione 1
4:          size[T] ← 1
5:          num[T] ← 0
6:      else if num[T] = size[T] then
7:          allocare new di dimensione 2 · size[T]
8:          copiare i num[T] elementi di Table[T] in new
9:          deallocare Table[T]
10:         Table[T] ← new
11:         size[T] ← 2 · size[T]
12:     end if
13:     inserire x in Table[T]
14:     num[T] ← num[T] + 1
15: end procedure
```

Il costo di un inserimento. Valutiamo i costi in termini di **inserimenti elementari**: costa 1 l'inserimento di x (riga 13) e costa 1 la copia di ciascun elemento durante un'espansione (riga 8), mentre il costo di allocazioni e deallocazioni è costante o comunque dominato da quello delle copie. Un inserimento che non provoca un'espansione costa quindi 1, mentre l'i-esimo inserimento, se provoca un'espansione, costa i: $i - 1$ copie degli elementi già presenti più l'inserimento del nuovo elemento. Partendo da una tabella vuota, la dimensione è sempre una potenza di 2, quindi

la tabella è piena prima dell'i-esimo inserimento esattamente quando $i-1$ è una potenza di 2:

$$c_i = \begin{cases} i & \text{se } i-1 \in 2^{\mathbb{N}} \\ 1 & \text{altrimenti} \end{cases}$$

dove $2^{\mathbb{N}} = \{1, 2, 4, 8, \dots\}$ è l'insieme delle potenze di 2. La tabella seguente mostra i primi nove inserimenti.

i	*num* prima	*size* prima	espansione	*size* dopo	c_i
1	0	0	(allocazione)	1	1
2	1	1	sì	2	2
3	2	2	sì	4	3
4	3	4	no	4	1
5	4	4	sì	8	5
6	5	8	no	8	1
7	6	8	no	8	1
8	7	8	no	8	1
9	8	8	sì	16	9

Analisi naive. Consideriamo $n = 2^k$ inserimenti a partire da una tabella vuota. L'inserimento di indice $2^{k-1}+1$ trova la tabella piena con 2^{k-1} elementi e costa

$$2^{k-1} + 1 = \frac{n}{2} + 1,$$

quindi il costo di un singolo inserimento nel caso peggiore è $O(n)$ e il costo di n inserimenti è $n \cdot O(n) = O(n^2)$. Come negli esempi precedenti, la stima è troppo pessimistica: le espansioni sono rare, e tra un'espansione e la successiva il numero di inserimenti economici raddoppia.

1.7.2 Analisi con l'aggregazione

Il calcolo è molto simile a quello dell'esercizio 1.5, con la differenza che le operazioni costose sono quelle di indice i con $i-1$ potenza di 2. Sia $n \geq 2$. Scriviamo il costo di ogni inserimento con espansione come $i = (i-1) + 1$, così da separare le copie dagli inserimenti veri e propri:

$$T(n) = \sum_{\substack{i=1 \\ i-1 \in 2^{\mathbb{N}}}}^{n} i + \sum_{\substack{i=1 \\ i-1 \notin 2^{\mathbb{N}}}}^{n} 1 = \sum_{\substack{i=1 \\ i-1 \in 2^{\mathbb{N}}}}^{n} (i-1) + \sum_{\substack{i=1 \\ i-1 \in 2^{\mathbb{N}}}}^{n} 1 + \sum_{\substack{i=1 \\ i-1 \notin 2^{\mathbb{N}}}}^{n} 1.$$

Le ultime due somme insieme contano tutte le n operazioni, mentre nella prima i valori $i-1$ sono le potenze di 2 comprese tra 1 e $n-1$, ovvero $2^0, 2^1, \dots, 2^{\lfloor \lg(n-1) \rfloor}$. Si ottiene quindi

$$T(n) = \sum_{j=0}^{\lfloor \lg(n-1) \rfloor} 2^j + n = 2^{\lfloor \lg(n-1) \rfloor + 1} - 1 + n \leq 2(n-1) - 1 + n < 3n$$

dove abbiamo usato $2^{\lfloor \lg(n-1) \rfloor} \leq n-1$. Pertanto

$$\text{costo ammortizzato per operazione} = \frac{T(n)}{n} < \frac{3n}{n} = 3 = O(1).$$

1.7.3 Analisi con gli accantonamenti

Il risultato dell'aggregazione suggerisce di addebitare 3 unità a ogni inserimento, e gli accantonamenti permettono di capire perché proprio 3. Poniamo

$$\hat{c}_{ins} = 3$$

e ripartiamo le tre unità in questo modo:

- **Inserimento**: 1 unità paga il costo reale dell'inserimento elementare dell'elemento.
- **Copia dell'elemento**: 1 unità viene accantonata sull'elemento stesso, e pagherà la sua copia alla prossima espansione.
- **Copia di un altro elemento**: 1 unità viene accantonata su un elemento già ricopiato in un'espansione precedente, che ha speso il proprio credito e non avrebbe altrimenti come pagare la copia successiva.

Il primo inserimento non trova alcun elemento già ricopiato da aiutare, quindi per esso possiamo porre $\hat{c}_{ins} = 2$. Supponiamo che subito dopo un'espansione la tabella abbia dimensione m: contiene allora $m/2$ elementi, tutti senza credito, perché lo hanno appena speso per farsi copiare. Prima della prossima espansione verranno inseriti altri $m/2$ elementi, ognuno dei quali accantona 2 unità, una su di sé e una su uno dei vecchi elementi: quando la tabella è di nuovo piena, ognuno degli m elementi porta con sé esattamente l'unità di credito che serve a copiarlo. La figura 1.5 mostra i primi sei inserimenti.

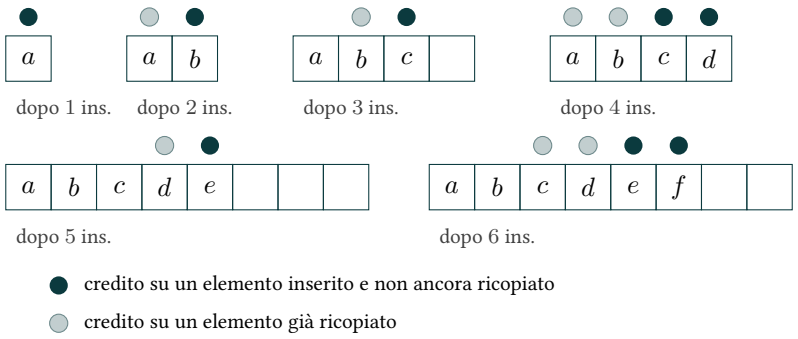


Figura 1.5. Crediti accantonati durante i primi sei inserimenti, con $\hat{c}_{ins} = 3$ (2 per il primo). Ogni elemento inserito dopo l'ultima espansione porta una moneta scura e ne deposita una chiara su un elemento già ricopiato. Dopo 4 inserimenti la tabella è piena e ognuno dei 4 elementi ha una moneta: sono proprio quelle che pagano le 4 copie dell'espansione provocata dal quinto inserimento, dopo la quale gli elementi a, b, c, d restano senza credito finché i nuovi inserimenti non ne depositano altro.

Il credito non è mai negativo. Verifichiamo ora formalmente che $B_j \geq 0$ per ogni j. Un inserimento senza espansione aumenta il credito di $\hat{c}_i - c_i = 3 - 1 = 2$, quindi l'unico momento in cui B_j potrebbe diventare negativo è subito dopo un'espansione, ovvero dopo un inserimento di indice $2^k + 1$, che porta la tabella dalla dimensione 2^k alla dimensione 2^{k+1}. Poiché $c_{2^k+1} = 2^k + 1$, si ha

$$B_{2^k+1} = B_{2^k} + \hat{c}_{2^k+1} - c_{2^k+1} = B_{2^k} + 3 - (2^k + 1) = B_{2^k} - 2^k + 2 \tag{1.2}$$

quindi basta dimostrare che $B_{2^k} \geq 2^k - 2$ per ogni k. Supponiamo per assurdo che non sia così, e sia m il più piccolo intero per cui

$$B_{2^m} < 2^m - 2.$$

Non può essere $m = 0$, perché $B_{2^0} = B_1 = \hat{c}_1 - c_1 = 2 - 1 = 1 > 2^0 - 2$; quindi $m > 0$ e, per la minimalità di m, vale $B_{2^{m-1}} \geq 2^{m-1} - 2$. Tra l'inserimento $2^{m-1} + 1$ e l'inserimento 2^m ci sono $2^{m-1} - 1$ inserimenti senza espansione, ognuno dei quali aggiunge 2 al credito, quindi usando la (1.2)

$$\begin{aligned} B_{2^m} &= B_{2^{m-1}+1} + 2\left(2^{m-1} - 1\right) = B_{2^{m-1}} - 2^{m-1} + 2 + 2^m - 2 \\ &= B_{2^{m-1}} + 2^{m-1} \geq 2^{m-1} - 2 + 2^{m-1} = 2^m - 2 \end{aligned}$$

contraddicendo l'ipotesi. Pertanto $B_j \geq 0$ per ogni $j \geq 1$ e, per il lemma 1.1,

$$T(n) = \sum_{i=1}^{n} c_i \leq \sum_{i=1}^{n} \hat{c}_i \leq 3n.$$

Il credito in funzione di num e $size$. Osservando la figura 1.5 si può esprimere il credito accumulato nella tabella usando soltanto $num[T]$ e $size[T]$. Dopo l'ultima espansione sono stati inseriti $num[T] - size[T]/2$ elementi, ognuno dei quali ha depositato una moneta su di sé e una su un elemento già ricopiato, quindi il credito totale è

$$B = 2\left(num[T] - \frac{size[T]}{2}\right) = 2 \cdot num[T] - size[T].$$

Ad esempio, dopo 6 inserimenti si ha $num = 6$ e $size = 8$, e infatti in figura ci sono $2 \cdot 6 - 8 = 4$ monete. La formula vale anche dopo il primo inserimento, che ha accantonato una sola unità: $2 \cdot 1 - 1 = 1$. Questa espressione dipende soltanto dalla configurazione della tabella, ed è il punto di partenza del metodo del potenziale.

1.7.4 Analisi con il potenziale

Usiamo come funzione potenziale proprio il credito appena trovato:

$$\Phi(T) = 2 \cdot num[T] - size[T].$$

Se T_0 è la tabella vuota si ha $\Phi(T_0) = 0$, e poiché la tabella è sempre piena almeno per metà, ovvero $\frac{1}{2} size[T] \leq num[T]$, vale

$$\Phi(T) \geq 0 = \Phi(T_0)$$

per ogni configurazione T. Il metodo del potenziale può quindi essere utilizzato per stimare i costi reali. Il potenziale si comporta esattamente come serve: vale 0 subito dopo un'espansione, quando $num[T] = size[T]/2$, e cresce di 2 a ogni inserimento fino a valere $size[T]$ quando la tabella è piena, cioè proprio il numero di elementi da copiare alla prossima espansione. La figura 1.6 mostra questo andamento.

Indichiamo con num_i e $size_i$ il numero di elementi e la dimensione della tabella dopo l'i-esimo inserimento, e con $\Phi_i = 2 \cdot num_i - size_i$ il potenziale corrispondente. In ogni caso $num_i = num_{i-1} + 1$, e distinguiamo tre casi.

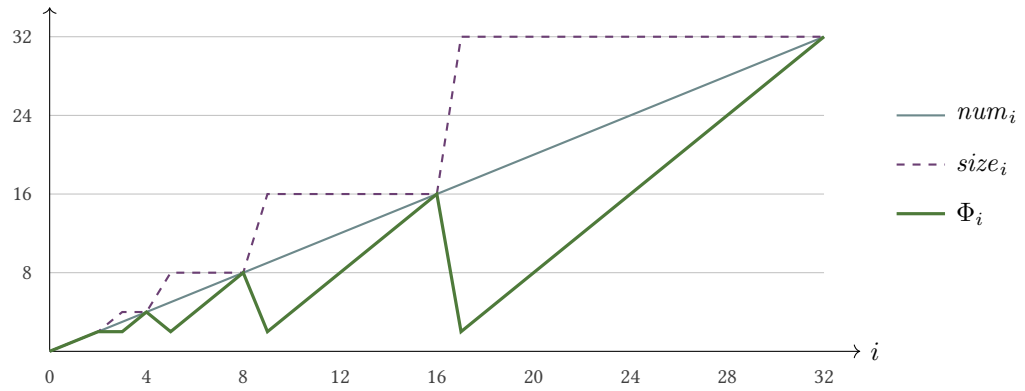


Figura 1.6. Andamento di num_i, $size_i$ e $\Phi_i = 2 \cdot num_i - size_i$ durante 32 inserimenti in una tabella inizialmente vuota, misurati dopo l'i-esimo inserimento. Subito prima di ogni espansione il potenziale ha raggiunto il numero di elementi presenti nella tabella, e può quindi pagare la loro copia; subito dopo scende a 0, ma l'inserimento che ha provocato l'espansione lo riporta a 2.

Inserimento senza espansione. La dimensione non cambia, ovvero $size_i = size_{i-1}$, e il costo reale è 1:

$$\begin{aligned}\hat{c}_{ins} &= 1 + (2 \cdot num_i - size_i) - (2 \cdot num_{i-1} - size_{i-1}) \\ &= 1 + 2(num_{i-1} + 1) - size_{i-1} - 2 \cdot num_{i-1} + size_{i-1} = 3.\end{aligned}$$

Inserimento con espansione in una tabella non vuota. La tabella era piena, quindi $size_{i-1} = num_{i-1}$, e dopo l'espansione $size_i = 2 \cdot size_{i-1} = 2 \cdot num_{i-1}$. Il costo reale è $num_i = num_{i-1} + 1$, ovvero num_{i-1} copie più l'inserimento:

$$\begin{aligned}\hat{c}_{ins} &= (num_{i-1} + 1) + \big(2(num_{i-1} + 1) - 2 \cdot num_{i-1}\big) - (2 \cdot num_{i-1} - num_{i-1}) \\ &= num_{i-1} + 1 + 2 - num_{i-1} = 3.\end{aligned}$$

Inserimento in una tabella vuota. È il primo inserimento ($i = 1$): si ha $size_1 = num_1 = 1$ e $\Phi_0 = 0$, quindi

$$\hat{c}_{ins} = 1 + (2 \cdot 1 - 1) - 0 = 2.$$

Ritroviamo così gli stessi costi degli accantonamenti, e poiché $\Phi_i \geq \Phi_0$

$$T(n) = \sum_{i=1}^{n} c_i \leq \sum_{i=1}^{n} \hat{c}_i \leq 3n.$$

Notiamo infine che, poiché la tabella è sempre piena almeno per metà, il fattore di carico non scende mai sotto $1/2$: lo spazio sprecato non supera mai la metà dello spazio allocato.

1.7.5 Inserimenti e cancellazioni

Per non sprecare memoria, quando si cancellano molti elementi conviene **contrarre** la tabella: si alloca una tabella più piccola, vi si copiano gli elementi rimasti e si libera quella vecchia. L'operazione Table-Delete(T, x) è analoga a Table-Insert: rimuove l'elemento, con costo 1, ed esegue una **contrazione** quando il fattore di carico diventa troppo basso. Una contrazione

costa quanto il numero di elementi da copiare, quindi una cancellazione che la provoca ha costo num_{i-1}, ovvero $num_{i-1} - 1$ copie più la cancellazione stessa.

Dimezzare a $1/2$ non funziona. La strategia più naturale sembra la simmetrica dell'espansione: raddoppiare la tabella quando è piena e dimezzarla quando il fattore di carico scende sotto $1/2$. In questo modo $\alpha(T)$ non scende mai sotto $1/2$, ma il costo ammortizzato può diventare molto alto. Consideriamo $n = 2^k$ operazioni: le prime $n/2$ sono inserimenti (I), che riempiono esattamente una tabella di dimensione $n/2$, e le restanti $n/2$ seguono lo schema I, C, C, I, I, C, C, I, I, …, dove C indica una cancellazione. La tabella seguente mostra cosa succede.

	I…I	I	C	C	I
$num[T]$	$n/2$	$n/2+1$	$n/2$	$n/2-1$	$n/2$
$size[T]$	$n/2$	n	n	$n/2$	$n/2$
$\alpha(T)$	1	$> 1/2$	$= 1/2$	< 1	1
c_i	$< 3n/2$ in totale	$n/2+1$	1	$n/2$	1

Il primo inserimento del secondo gruppo provoca un'espansione, la prima cancellazione riporta il fattore di carico esattamente a $1/2$ e la seconda lo fa scendere sotto, provocando una contrazione; dopo il successivo inserimento la tabella è di nuovo piena, e l'inserimento seguente provoca un'altra espansione. Ogni gruppo di quattro operazioni contiene quindi un'espansione e una contrazione, che costano circa $n/2$ ciascuna, e i gruppi sono $n/8$: il costo totale è dell'ordine di $\frac{n}{8} \cdot n = \frac{n^2}{8}$, ovvero $\Theta(n^2)$, e il costo ammortizzato per operazione è $\Theta(n)$. Il problema è che subito dopo un'espansione non si cancellano abbastanza elementi da pagare una contrazione, e subito dopo una contrazione non se ne inseriscono abbastanza da pagare un'espansione.

Dimezzare a $1/4$. La soluzione è lasciare più margine: si continua a raddoppiare la tabella quando è piena, ma la si dimezza soltanto quando una cancellazione farebbe scendere il fattore di carico sotto $1/4$. In altre parole ci accontentiamo di avere

$$\alpha(T) \geq \frac{1}{4}$$

in cambio di un costo ammortizzato costante.[4] Sia dopo un'espansione sia dopo una contrazione il fattore di carico torna circa a $1/2$, e serve un numero di operazioni proporzionale a $num[T]$ per arrivare alla riallocazione successiva, in un senso o nell'altro.

Ci serve quindi un potenziale che valga 0 quando $\alpha(T) = 1/2$ e cresca man mano che il fattore di carico si allontana da $1/2$, in modo da avere accumulato abbastanza potenziale da pagare la copia di tutti gli elementi quando $\alpha(T)$ arriva a 1 oppure scende a $1/4$. Poniamo

$$\Phi(T) = \begin{cases} 2 \cdot num[T] - size[T] & \text{se } \alpha(T) \geq \dfrac{1}{2} \\ \dfrac{1}{2} size[T] - num[T] & \text{se } \alpha(T) \leq \dfrac{1}{2}. \end{cases}$$

Le due espressioni coincidono quando $\alpha(T) = 1/2$: in quel caso $num[T] = \frac{1}{2} size[T]$, quindi $\frac{1}{2} size[T] - num[T] = 0$ e $2 \cdot num[T] - size[T] = 0$. Questo ci permette di usare l'una o l'altra, a

[4] Per semplicità trascuriamo il caso limite in cui la tabella si svuota del tutto; nel libro si assume che in quel caso la tabella venga deallocata, ponendo $size[T] = 0$.

seconda di cosa è più comodo, quando la tabella è piena esattamente per metà. Inoltre vale sempre $\Phi(T) \geq 0 = \Phi(T_0)$, perché ciascuna delle due espressioni è non negativa nel proprio intervallo. La figura 1.7 mostra l'andamento del potenziale in funzione del fattore di carico.

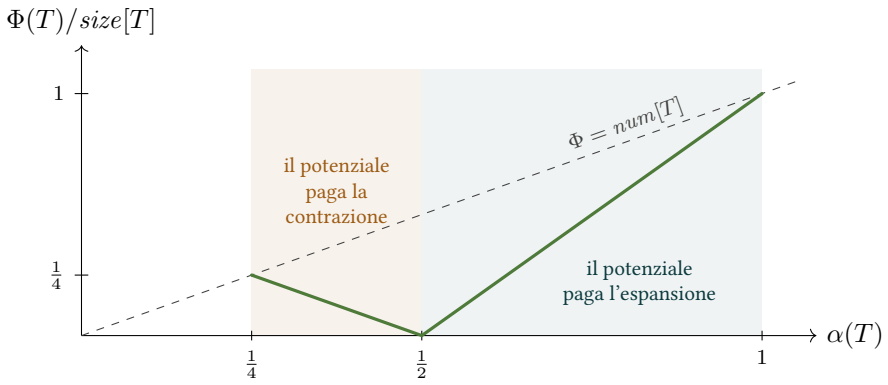


Figura 1.7. Potenziale della tabella, in rapporto alla sua dimensione, in funzione del fattore di carico. Il potenziale è nullo quando la tabella è piena per metà e cresce linearmente in entrambe le direzioni: quando $\alpha(T) = 1$ il potenziale vale $size[T]$, cioè $num[T]$, e quando $\alpha(T) = 1/4$ vale $\frac{1}{4} size[T]$, cioè ancora $num[T]$. In entrambi i casi tocca la retta tratteggiata $\Phi = num[T]$, cioè è sufficiente a pagare la copia di tutti gli elementi.

Indichiamo come prima con num_i e $size_i$ i valori dopo l'i-esima operazione e con $\alpha_{i-1} = \alpha(T_{i-1})$ il fattore di carico prima dell'operazione, e calcoliamo i costi ammortizzati distinguendo i casi.

Analisi degli inserimenti. In ogni caso $num_i = num_{i-1} + 1$.

- **Caso** $\alpha_{i-1} \geq 1/2$: anche dopo l'inserimento la tabella è piena almeno per metà, quindi si usa la prima espressione del potenziale sia prima sia dopo. Il calcolo è lo stesso già fatto nel caso dei soli inserimenti, e dà $\hat{c}_{ins} = 3$ (oppure 2 per il primo inserimento).
- **Caso** $1/4 \leq \alpha_{i-1} < 1/2$: non c'è espansione, quindi $size_i = size_{i-1}$, e poiché $num_{i-1} < \frac{1}{2} size_{i-1}$ si ha $num_i \leq \frac{1}{2} size_i$, ovvero $\alpha_i \leq 1/2$. Si usa allora la seconda espressione:

$$\begin{aligned} \hat{c}_{ins} &= 1 + \left(\tfrac{1}{2} size_i - num_i\right) - \left(\tfrac{1}{2} size_{i-1} - num_{i-1}\right) \\ &= 1 + \tfrac{1}{2} size_{i-1} - num_{i-1} - 1 - \tfrac{1}{2} size_{i-1} + num_{i-1} = 0. \end{aligned}$$

Un inserimento in una tabella piena per meno di metà ha costo ammortizzato nullo: il suo costo reale è pagato dalla diminuzione del potenziale, perché avvicina il fattore di carico a $1/2$.

Analisi delle cancellazioni. In ogni caso $num_i = num_{i-1} - 1$.

- **Caso** $1/2 < \alpha_{i-1} \leq 1$: non c'è contrazione e dopo la cancellazione la tabella è ancora piena almeno per metà, quindi si usa la prima espressione con $size_i = size_{i-1}$:

$$\begin{aligned} \hat{c}_{canc} &= 1 + (2 \cdot num_i - size_i) - (2 \cdot num_{i-1} - size_{i-1}) \\ &= 1 + 2 \cdot num_{i-1} - 2 - size_{i-1} - 2 \cdot num_{i-1} + size_{i-1} = -1. \end{aligned}$$

- **Caso** $1/4 < \alpha_{i-1} \leq 1/2$: non c'è contrazione e dopo la cancellazione $\alpha_i < 1/2$, quindi si usa la seconda espressione con $size_i = size_{i-1}$:

$$\begin{aligned}\hat{c}_{canc} &= 1 + \left(\tfrac{1}{2} size_i - num_i\right) - \left(\tfrac{1}{2} size_{i-1} - num_{i-1}\right) \\ &= 1 + \tfrac{1}{2} size_{i-1} - num_{i-1} + 1 - \tfrac{1}{2} size_{i-1} + num_{i-1} = 2.\end{aligned}$$

- **Caso** $\alpha_{i-1} = 1/4$: la cancellazione provoca una contrazione. Si ha $num_{i-1} = \frac{1}{4} size_{i-1}$ e $size_i = \frac{1}{2} size_{i-1}$, e il costo reale è num_{i-1}. Anche dopo la contrazione $\alpha_i < 1/2$, quindi

$$\begin{aligned}\hat{c}_{canc} &= num_{i-1} + \left(\tfrac{1}{2} size_i - num_i\right) - \left(\tfrac{1}{2} size_{i-1} - num_{i-1}\right) \\ &= \tfrac{1}{4} size_{i-1} + \left(\tfrac{1}{4} size_{i-1} - \tfrac{1}{4} size_{i-1} + 1\right) - \left(\tfrac{1}{2} size_{i-1} - \tfrac{1}{4} size_{i-1}\right) \\ &= \tfrac{1}{4} size_{i-1} + 1 - \tfrac{1}{4} size_{i-1} = 1.\end{aligned}$$

Riassumendo. I costi ammortizzati delle due operazioni sono

$$\hat{c}_{ins} = \begin{cases} 0 & \text{se } \frac{1}{4} \leq \alpha(T) < \frac{1}{2} \\ 3 & \text{se } \frac{1}{2} \leq \alpha(T) \leq 1 \end{cases} \qquad \hat{c}_{canc} = \begin{cases} -1 & \text{se } \frac{1}{2} < \alpha(T) \leq 1 \\ 2 & \text{se } \frac{1}{4} < \alpha(T) \leq \frac{1}{2} \\ 1 & \text{se } \alpha(T) = \frac{1}{4} \end{cases}$$

dove $\alpha(T)$ è il fattore di carico prima dell'operazione. Pertanto $\hat{c}_{ins} \leq 3$ e $\hat{c}_{canc} \leq 2 < 3$, e poiché $\Phi(T_i) \geq 0 = \Phi(T_0)$ si ottiene

$$T(n) = \sum_{i=1}^{n} c_i \leq \sum_{i=1}^{n} \hat{c}_i \leq 3n.$$

Qualsiasi sequenza di n inserimenti e cancellazioni su una tabella inizialmente vuota costa quindi $O(n)$, e il costo ammortizzato di ciascuna operazione è $O(1)$.

In sintesi, raddoppiare la tabella quando è piena garantisce un costo ammortizzato costante per gli inserimenti e uno spreco di memoria al più pari a metà dello spazio allocato; dimezzarla appena scende sotto metà sembra la scelta simmetrica, ma porta a sequenze di costo $\Theta(n^2)$; dimezzarla soltanto sotto $1/4$ mantiene il costo ammortizzato costante anche con le cancellazioni, al prezzo di uno spreco di memoria che può arrivare a tre quarti dello spazio allocato.

Riferimenti

Il materiale visto in questo capitolo comprende:

- Materiale visto a lezione.
- Capitolo 17 (*Amortized Analysis*, sezioni 17.1–17.4) del libro T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*.